BreastCytoAI Documentation

An AI-powered app that predicts benign and malignant breast cancer cells using multiple features and models.

Features

- 🖼️ **Image Viewer**: Supports PNG, JPG, BMP, TIFF medical images
- ✂️ **Image Cropping**: ROI selection with automatic padding
- 🧠 **Multi-Model Support**:
  - Radiomics feature models (texture, shape, intensity features)
  - Pixel-level models (ResNet, DenseNet, EfficientNet, MobileNet, etc.)
  - SIFT feature models (scale-invariant keypoints and descriptors)
  - Fusion models (Radiomics + Pixel, SIFT + Pixel)
- 📊 **Visualization Analysis**:
  - GradCAM heatmaps (shows model attention regions)
  - Occlusion sensitivity maps (shows critical regions)
- 📋 **Clinical Reports**: Detailed prediction reports with clinical recommendations
- 🎨 **Modern Interface**: Professional medical application design with adaptive layouts

System Requirements

- Python 3.8+
- Recommended: 8GB+ RAM, CUDA-compatible GPU (optional)

Installation and Running

Prerequisites

- Python 3.8+ (managed via conda)
- Git
- Recommended: 8GB+ RAM, CUDA-compatible GPU (optional)

Step 1: Clone the Repository

```
# Clone the project repository
git clone https://github.com/RogueLiquid/BIA4-BreastCytoAI-Group6
cd BIA4-BreastCytoAI-Group6
```

Step 2: Models Preparation

To get models for this tool please download from: https://zenodo.org/records/17921766

Please move the decompressed `\Model` directory under `\BreastCytoAI`.

Step 3: Environment Setup with Conda

```
# 1. Create a new conda environment
conda create -n breastcytoai python=3.8 -y

# 2. Activate the environment
conda activate breastcytoai

# 3. Install dependencies
pip install -r requirements.txt
```

Tool Usage Method 1: Run as Python Application (Recommended)

```
# Ensure you're in the project directory and environment is activated
cd BreastCytoAI
conda activate breastcytoai

# Run the application
python gui.py
```

Tool Usage Method 2: Package as Standalone Executable

Cross-Platform Automated Build

```
# Ensure environment is activated
conda activate breastcytoai

# Use the automated build script
python build_exe.py
```

After building, the executable is located at `dist/BreastCancerClassifier`

Windows-Specific Build

```
# Activate conda environment first
conda activate breastcytoai

# Double-click or run in command prompt
build_windows.bat
```

macOS/Linux-Specific Build

```
# Activate conda environment first
conda activate breastcytoai

# Run the build script
./build_unix.sh
```

Manual Build (Advanced Users)

```
# 1. Ensure environment is activated
conda activate breastcytoai

# 2. Install PyInstaller (if not already installed)
pip install pyinstaller

# 3. Build executable
pyinstaller --onefile --windowed --name BreastCancerClassifier gui.py

# 4. Add model files
# Windows:
pyinstaller --onefile --windowed --add-data "ml_backend.py;." --add-data "models;models" gui.py

# macOS/Linux:
pyinstaller --onefile --windowed --add-data "ml_backend.py:." --add-data "models:models" gui.py
```

Environment Cleanup

After using the application, you can deactivate the conda environment:

```
# Deactivate the environment
conda deactivate
```

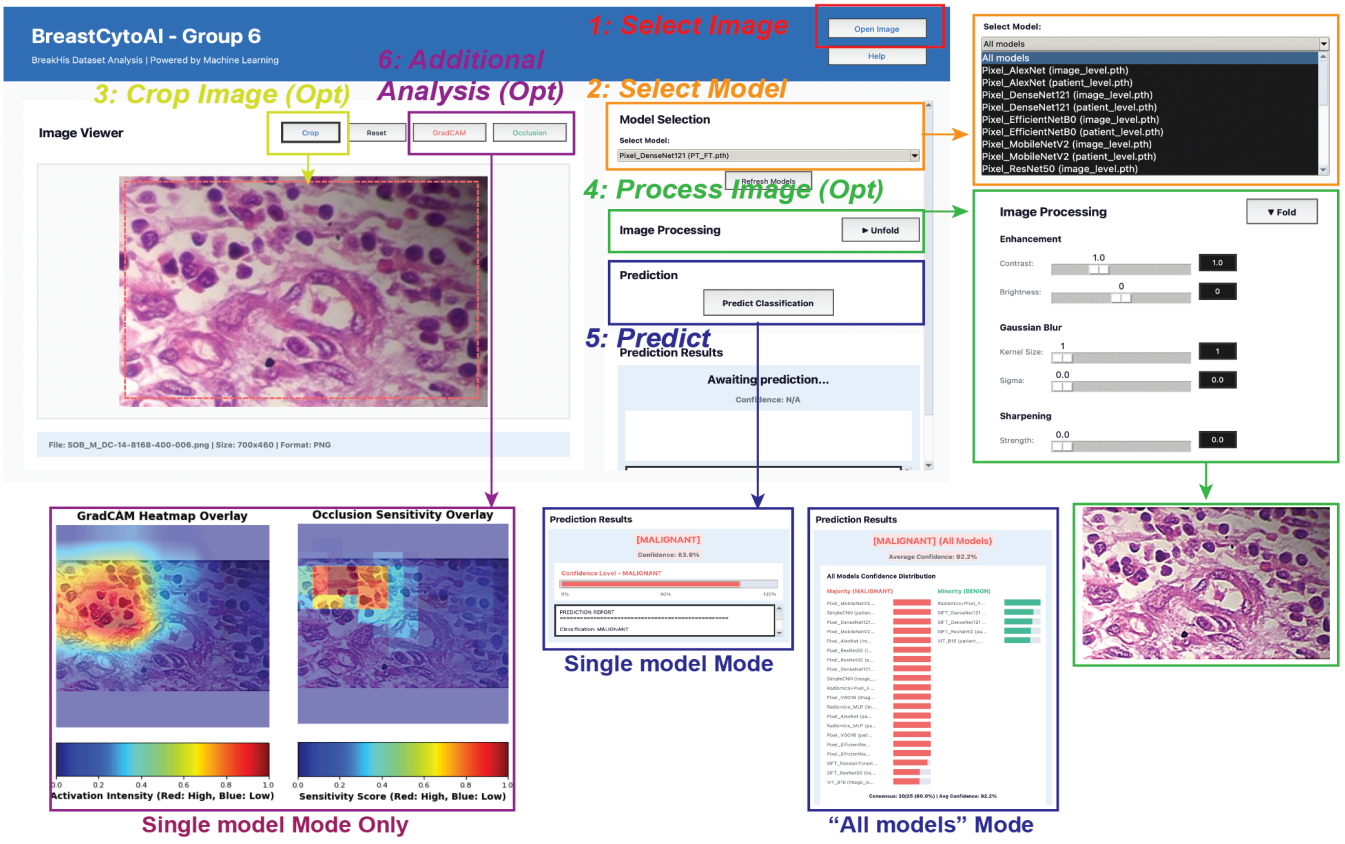
```
# Optional: Remove the environment if no longer needed
conda env remove -n breastcytoai
```

Project Structure

```
├── gui.py                  # Main GUI application
├── ml_backend.py           # Machine learning backend
├── requirements.txt        # Python dependencies
├── build_exe.py           # Automated build script
├── build_windows.bat      # Windows build script
├── build_unix.sh          # macOS/Linux build script
├── README.md              # This documentation
└── [model folders]/       # Trained model files
    ├── model (.pth)/
    ├── other info/
    └── ...
```

Usage Instructions

Basic Workflow



- **Launch Application**: Run `python gui.py` or double-click the executable
- **Load Image**: Click "Open Image" to select a medical image
- **Select Model**: Choose a pretrained model from the dropdown menu
- **Optional Operations**:
  - Use "Crop" tool to select region of interest
  - Use "Image Processing" to apply enhancement, blur and sharpening
  - Use "Reset" to restore original image
- **Predict Classification**: Click "Predict Classification"
- **Visualization Analysis**:
  - Click "GradCAM" to view model attention regions
  - Click "Occlusion" to view critical region sensitivity

Model Type Descriptions

- **Radiomics Models**: MLP models based on 39 radiomics features (texture, shape, intensity statistics)
- **Pixel Models**: CNN architectures (ResNet, DenseNet, EfficientNet, MobileNet, VGG) that process images directly
- **SIFT Models**: Traditional ML models (Random Forest) using SIFT keypoints and Bag-of-Features descriptors
- **Fusion Models**: Multi-modal approaches combining:
  - Radiomics + CNN features
  - SIFT + CNN features
- **All Models**: Ensemble prediction using all available models with majority voting

Technical Details

Dependencies

- **PyTorch**: Deep learning framework with torchvision
- **OpenCV**: Image processing and computer vision
- **Pillow**: Image loading and manipulation
- **scikit-image**: Radiomics feature extraction (GLCM, GLRLM, etc.)
- **scikit-learn**: Traditional ML algorithms and clustering
- **matplotlib**: Visualization and plotting
- **tkinter**: GUI framework (built-in Python)
- **numpy**: Numerical computing

Feature Extraction

- **Radiomics Features**: 39-dimensional feature vector including:
  - GLCM (Gray Level Co-occurrence Matrix) statistics
  - GLRLM (Gray Level Run Length Matrix) statistics
  - GLSZM (Gray Level Size Zone Matrix) statistics
  - Shape and intensity features
- **SIFT Features**: Scale-invariant keypoints with Bag-of-Features (500-dimensional)
- **Pixel Features**: Raw RGB pixel values for MLP models

Model Architectures

- **RadiomicsMLP**: 39-dimensional input → 128 → 64 → 2-class classification
- **CNN Models**: Multiple architectures (ResNet, DenseNet, EfficientNet, MobileNet, VGG)
- **SIFT Models**: Random Forest on 500-dimensional BoF features
- **Fusion Models**:
  - Radiomics + CNN: Concatenated features through adaptive classifier
  - SIFT + CNN: Multi-modal fusion with hybrid architecture
- **Ensemble**: Majority voting across all available models

Troubleshooting

Common Issues

1. **Model Loading Failure**
   - Ensure .pth files are in correct locations: Make sure each .pth file is in the corresponding naming folder (such as Pixel_ResNet50), as our tool construct models by reconizing the folder name.
   - Check file permissions
2. **CUDA Errors**
   - Install CUDA version of PyTorch: `pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118`
3. **Memory Insufficient**
   - Use smaller models (VGG and Vit are larger)
   - Close unnecessary applications
4. **Executable Build Failure**
   - Ensure all dependencies are installed

- Check disk space (requires 2-3GB)

Performance Optimization

- Use GPU acceleration (if available)
- Choose appropriate model sizes
- Regularly clean temporary files

License

This project is for educational and research purposes only.

Contributing

Issues and improvement suggestions are welcome!

Version History

- **v3.0**: Added All models mode
- **v2.0**: Added Occlusion visualization, improved interface, enhanced clinical reports
- **v1.0**: Basic functionality implementation